

Part 1: Agentic Banking POC Lifecycle

1. Purpose of this POC

We are building an HDFC-like Smart Banking Assistant as an Agentic AI proof of concept.

The assistant supports three high-level problem areas:

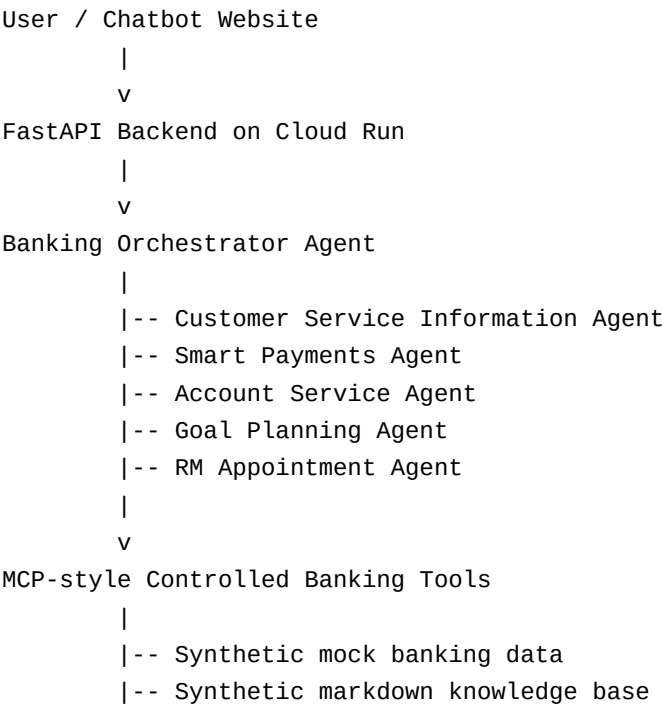
- 1. Customer service information
- 2. Banking assistance
- 3. Relationship Manager appointment booking

The banking assistance area currently includes:

- Smart payments
- Account service
- Goal planning

This is a safe POC. It uses synthetic data and mock banking tools. No real payment, real customer data, or real banking action is performed.

2. Current architecture



The important design idea is that the model or agent does not directly manipulate banking

data. It calls controlled tools.

3. Repository branch used

Feature branch:

cursor/agentik-banking-poc-88e6

In Cloud Shell, we used:

```
git clone https://github.com/anantjujgar111/Agentik-Ai-Project.git
cd Agentik-Ai-Project
git checkout cursor/agentik-banking-poc-88e6
```

If the repo was nested after cloning, the working path became:

```
cd ~/Agentik-Ai-Project/Agentik-Ai-Project
```

4. Backend API created

Main file:

backend/app/main.py

What it provides:

```
GET /health
POST /chat
```

The '/chat' endpoint receives a user message and sends it to the orchestrator.

Example request:

```
{
  "user_id": "cust_001",
  "message": "Show my balance"
```

```
}
```

5. Orchestrator agent created

Main file:

`backend/app/agents/orchestrator.py`

The orchestrator checks the user message and routes it to the right agent flow.

Examples:

"Show my balance" -> Account Service Agent

"Schedule my electricity bill" -> Smart Payments Agent

"Create wedding goal for 5 lakh in 18 months" -> Goal Agent

"Book RM appointment" -> RM Appointment Agent

"What is debit card annual fee?" -> Customer Service Agent

6. Agent flows created

All current agent flows are in:

`backend/app/agents/orchestrator.py`

Important functions:

`_customer_service()`

`_payments()`

`_account_service()`

`_goal()`

`_rm_appointment()`

Customer Service Information Agent

Uses local markdown knowledge-base search.

Related files:

backend/app/services/knowledge.py
data/knowledge_base/

Current behavior:

User question

-> Search markdown knowledge base

-> Return matching snippets

-> Optionally polish response with Vertex AI Gemini if USE_VERTEX=true

Smart Payments Agent

Handles:

- Due bill lookup
- Payment rail recommendation
- Mock payment scheduling
- Rent split preview

Main tool file:

backend/app/mcp/tools.py

Main mock data file:

data/mock/bills.json

Example flow:

User: Schedule my electricity bill

-> list_due_bills()

-> select electricity bill

-> recommend_payment_rail()

-> schedule_payment()

-> return response with tool traces

Account Service Agent

Handles:

- Balance queries
- Statement summaries
- Charge explanation
- Mock card control updates

Related files:

backend/app/mcp/tools.py
data/mock/accounts.json
data/mock/transactions.json
data/mock/cards.json

Goal Planning Agent

Handles goals like:

- Emergency fund
- Vacation
- Education
- Wedding
- Home down payment
- Retirement

The current goal plan is illustrative and includes a disclaimer.

Related file:

backend/app/mcp/tools.py

RM Appointment Agent

Handles:

- Listing RM slots
- Booking a mock RM appointment

Related files:

backend/app/mcp/tools.py
data/mock/rm_slots.json

7. MCP-style tool boundary created

Main file:

backend/app/mcp/tools.py

This file contains controlled functions such as:

```
get_balance()  
get_statement_summary()  
explain_transaction()  
list_due_bills()  
recommend_payment_rail()  
schedule_payment()  
split_rent()  
create_goal_plan()  
list_rm_slots()  
book_rm_appointment()
```

Why this matters:

```
Agent decides what is needed  
-> Tool performs controlled action  
-> Tool returns structured result  
-> Agent responds to user
```

This is safer than allowing the LLM to directly read or modify banking records.

8. Synthetic mock data created

Mock data folder:

data/mock/

Files:

```
data/mock/customers.json  
data/mock/accounts.json  
data/mock/transactions.json
```

data/mock/bills.json
data/mock/cards.json
data/mock/beneficiaries.json
data/mock/fd_rd.json
data/mock/rm_slots.json

Example mock customer:

cust_001 -> Aarav Mehta

This data lets the chatbot behave like a real banking assistant without using real customer data.

9. Synthetic knowledge base created

Knowledge-base folder:

data/knowledge_base/

Files:

data/knowledge_base/customer_service/savings_account_faq.md
data/knowledge_base/customer_service/charges_and_fees.md
data/knowledge_base/customer_service/debit_card_controls.md
data/knowledge_base/banking_products/fd_rd_info.md
data/knowledge_base/banking_products/payment_methods.md
data/knowledge_base/policies/beneficiary_and_limits.md

These files are written as synthetic bank FAQ, product, and policy documents.

10. Vertex AI model integration status

File:

backend/app/services/vertex_client.py

Current model setting:

VERTEX_MODEL=gemini-1.5-flash

Current behavior:

If USE_VERTEX=false:

deterministic local response is returned

If USE_VERTEX=true:

Gemini on Vertex AI is used to polish the response text

Environment variable used:

export USE_VERTEX=true

11. ADK integration status

File:

backend/app/agents/adk_agent.py

This file marks the Google ADK integration point.

Current status:

The runnable POC uses the deterministic orchestrator.

The ADK agent factory exists as the next integration step.

Reason:

First prove the banking tool flow end-to-end.

Then deepen the ADK runtime integration.

12. Frontend chatbot created

Frontend folder:

frontend/

Files:

```
frontend/index.html
frontend/app.js
frontend/styles.css
frontend/config.js
```

The frontend is a static chatbot UI. It will be hosted later using Firebase Hosting.

Backend URL config file:

```
frontend/config.js
```

This value must point to Cloud Run:

```
window.CHAT_API_BASE_URL = "https://hdfc-banking-api-33622648493.us-central1.run.app";
```

13. GCP values used

Project:

```
export GCP_PROJECT_ID=project-687fc909-6531-45c6-950
```

Region:

```
export GCP_REGION=us-central1
```

Knowledge bucket:

```
export GCS_KNOWLEDGE_BUCKET=hdfc-agentic-kb-project-687fc909-6531-45c6-950
```

Vertex enabled:

```
export USE_VERTEX=true
```

14. GCP APIs enabled

Command used:

```
gcloud services enable \
  aiplatform.googleapis.com \
  run.googleapis.com \
  artifactregistry.googleapis.com \
  cloudbuild.googleapis.com \
  storage.googleapis.com \
  discoveryengine.googleapis.com
```

What each API is for:

aiplatform.googleapis.com	-> Vertex AI / Gemini
run.googleapis.com	-> Cloud Run backend hosting
artifactregistry.googleapis.com	-> Container image storage
cloudbuild.googleapis.com	-> Build container from source
storage.googleapis.com	-> Cloud Storage bucket for knowledge base
discoveryengine.googleapis.com	-> Vertex AI Search / Agent Builder indexing

15. Knowledge base uploaded to GCS

Script:

```
scripts/upload_knowledge_to_gcs.sh
```

Command used:

```
bash scripts/upload_knowledge_to_gcs.sh "$GCP_PROJECT_ID" "$GCS_KNOWLEDGE_BUCKET"
```

Uploaded GCS path:

gs://hdfc-agentic-kb-project-687fc909-6531-45c6-950/knowledge_base

Meaning:

The markdown files are now stored in Cloud Storage.
They are ready to be imported into Vertex AI Search later.

16. Backend deployed to Cloud Run

Script:

scripts/deploy_backend_cloud_run.sh

Command used:

```
export GCP_REGION=us-central1
export USE_VERTEX=true
bash scripts/deploy_backend_cloud_run.sh "$GCP_PROJECT_ID"
```

Cloud Run service:

hdfc-banking-api

Cloud Run backend URL:

<https://hdfc-banking-api-33622648493.us-central1.run.app>

Deployment result:

Backend is deployed and serving 100 percent traffic.

17. Backend test commands for next step

Health check:

```
curl https://hdfc-banking-api-33622648493.us-central1.run.app/health
```

Expected kind of response:

```
{
  "status": "ok",
  "service": "HDFC Smart Banking Assistant POC"
}
```

Balance query:

```
curl -X POST https://hdfc-banking-api-33622648493.us-central1.run.app/chat \
  -H "Content-Type: application/json" \
  -d '{"user_id":"cust_001","message":"Show my balance"}'
```

Payment query:

```
curl -X POST https://hdfc-banking-api-33622648493.us-central1.run.app/chat \
  -H "Content-Type: application/json" \
  -d '{"user_id":"cust_001","message":"Schedule my electricity bill"}'
```

Goal query:

```
curl -X POST https://hdfc-banking-api-33622648493.us-central1.run.app/chat \
  -H "Content-Type: application/json" \
  -d '{"user_id":"cust_001","message":"Create wedding goal for 5 lakh in 18 months"}'
```

RM appointment query:

```
curl -X POST https://hdfc-banking-api-33622648493.us-central1.run.app/chat \
  -H "Content-Type: application/json" \
  -d '{"user_id":"cust_001","message":"Book RM appointment"}'
```

18. What is completed in Part 1

Completed:

- Backend API created
- Agent orchestrator created
- Specialized agent flows created
- MCP-style banking tools created
- Synthetic mock data created
- Synthetic knowledge base created
- Knowledge base uploaded to Cloud Storage
- Cloud Run backend deployed
- Frontend chatbot files created
- Deployment scripts created
- Lifecycle and GCP setup docs created

19. What remains for Part 2

Next items:

- Test live backend endpoints
- Connect frontend/config.js to Cloud Run URL
- Deploy frontend to Firebase Hosting
- Create or import Vertex AI Search data store
- Connect code to Vertex AI Search retrieval
- Deepen ADK runtime integration
- Optionally separate MCP as its own service
- Add authentication/session handling for a stronger banking demo

20. Interview explanation for Part 1

You can explain Part 1 like this:

In Part 1, I built the foundation of an Agentic AI banking assistant. The backend runs on FastAPI and Cloud Run. The orchestrator routes user requests to specialized banking agents. Sensitive banking functions are exposed as controlled MCP-style tools. The POC uses synthetic customer, account, transaction, bill, card, FD/RD, and RM appointment data. The knowledge base is written as markdown documents, uploaded to Cloud Storage, and prepared for Vertex AI Search indexing. Vertex AI Gemini can polish responses when USE_VERTEX=true. The frontend chatbot is ready to connect to the Cloud Run backend and deploy to

Firebase Hosting.